

COMPREHENSIVE
LABORATORY ACTIVITY

BCIS Subscription Billing and Collection System
Subscription Billing, Collection, and Subscriber Ledger System

A comprehensive desktop application laboratory project for Bukidnon Cable and Internet Services

Project Context

Design and implement a professional Windows desktop system for three simultaneous office PCs. The solution must support Internet, Cable and Combo subscriptions, monthly billing, subscriber ledgers, Cash/GCash collection, collector accountability, overdue receivables, reports, audit trails and reliable backup/restore.

Project Information	Details
Activity Type	Comprehensive multi-session laboratory project
Target Environment	3 office Windows PCs connected to a local area network (LAN)
Architecture	Electron desktop clients → Fastify API → PostgreSQL
Development Style	Incremental, test-driven for financial rules, AI-assisted development permitted under accountability rules
Assessment	100 points + required functional acceptance tests
Primary Output	Working application, source code, database, technical documentation, test evidence and presentation/demo

1. Laboratory Activity Overview

This laboratory activity presents a practical software-engineering project. The implementation must analyze the business process, apply a secure client-server architecture, support transaction-safe billing and collection workflows, verify financial correctness, and deliver a professional desktop application.

1.1 Business Scenario

Bukidnon Cable and Internet Services provides Internet and Cable subscription services. The company needs a system that can monitor subscriber accounts, monthly charges, payment history, house-to-house collections, GCash payments, overdue balances, suspensions, reconnections, and management reports. Three office computers must be able to use the system simultaneously.

Subscribers may pay at the office, through a field collector, or through GCash. GCash customers currently send proof of successful transactions to the company Facebook Page. Staff must verify and record those payments. The owner also requires a clear view of accounts receivable, overdue subscribers, monthly collection performance, collector remittances, and the complete ledger history of each subscriber.

1.2 Laboratory Challenge



Core challenge

Build a system that is not merely CRUD. The submission must preserve financial history, support concurrent office users, correctly allocate payments, reconcile collector cash, produce audit-ready reports, and protect the database from unsafe direct access.

1.3 General Objective

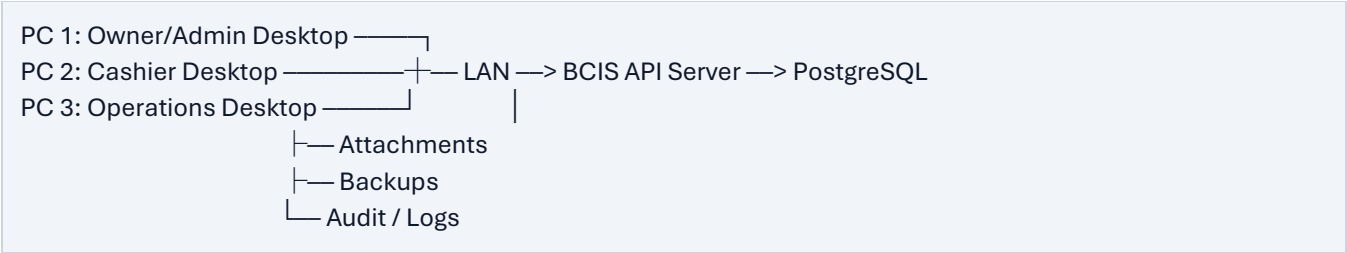
Design, develop, test, document, and demonstrate the BCIS Subscription Billing and Collection System for subscription billing, collection monitoring, subscriber ledger management, accounts receivable monitoring, and operational reporting.

1.4 Specific Learning Outcomes

- Analyze a real-world subscription and collection workflow and translate it into system requirements.
- Design a client-server desktop architecture appropriate for three concurrent office PCs.
- Model subscribers, service accounts, plans, invoices, payments, payment allocations, collectors, remittances, and audit records using PostgreSQL.
- Implement secure typed communication between Electron clients and a central API.
- Apply reliable financial rules for money values, partial payments, advance payments, reversals, and ledger balances.
- Create a professional, modern and consistent desktop UI/UX for dense operational workflows.
- Implement RBAC, validation, auditing, logging, backup, and recovery safeguards.
- Create PDF/XLSX management reports and subscriber statements.
- Write unit, integration and end-to-end tests for critical financial workflows.
- Produce complete technical documentation and defend the implementation during a final demonstration.

2. Required Technology and Architecture

2.1 Required Production Architecture



The three office computers are application clients. They must not open a shared database file directly. All operational data access passes through the API server so transactions, authorization, validation, and audit behavior remain consistent.

2.2 Recommended Technology Stack

Layer	Technology	Purpose
Desktop shell	Electron 44.x + electron-vite + electron-builder	Windows desktop application and installer
UI	React 19.2 + TypeScript strict	Typed desktop UI
Styling	Tailwind CSS 4 + shadcn/ui	Professional consistent design system
Tables / data	TanStack Table + TanStack Query	Dense operational tables and server state
API	Fastify 5 + TypeScript	Central LAN API for all three PCs
Database	PostgreSQL	Concurrent transactional database
ORM	Drizzle ORM	Typed schema and migrations
Validation	Zod 4	Shared request/response validation
Forms	React Hook Form	Validated business forms
Reports	ExceUS + pdfmake	XLSX and PDF outputs
Testing	Vitest + API integration tests + Playwright Electron	Domain, integration and E2E testing
Logging	Pino	Structured application/server logs

2.3 Architectural Rules

- The Electron renderer must never connect directly to PostgreSQL.
- Use contextIsolation=true and nodeIntegration=false.
- Expose only a narrow typed preload API to the renderer.
- Validate all external inputs using Zod.

- Keep business rules in services/domain logic, not in React components.
- Use PostgreSQL transactions for multi-step financial posting operations.
- Use integer centavos or exact NUMERIC/DECIMAL handling for money; never JavaScript floating-point arithmetic for authoritative monetary calculations.
- Every database schema change must be implemented through a migration.
- Posted financial records are reversed or adjusted; they are not silently deleted or overwritten.

3. Functional Scope and System Requirements

3.1 User Roles and Access

Role	Minimum Capabilities
Owner / Super Admin	Full dashboards, reports, approvals, configuration, users, audit, backup/restore
Administrator	Subscribers, plans, service accounts, billing, collections, operational reports
Cashier	Subscriber search, receive payment, issue receipt, view balances, approved GCash processing
Collection Supervisor	Collection areas/routes, batches, remittance, reconciliation, collector performance
Accounting / Auditor	Reports, adjustments/reversals review, receivables, audit trails
Technician	Service account and suspension/reconnection operational information only
Read-only Viewer	Dashboards and reports without mutation rights

3.2 Subscriber Management

- Create and maintain subscriber identity, account number, contacts, address, collection area, assigned collector, billing/due day, status and notes.
- Allow one subscriber to own multiple service accounts and/or service addresses.
- Provide global search by account number, name, contact number, address, receipt number, invoice number and GCash reference.
- Do not delete historical subscriber accounts. Use inactive/terminated/archived statuses.

3.3 Plans and Service Accounts

- Support Internet, Cable and Combo plans with code, name, price, fees, description and active status.
- Internet plans may include speed attributes; Cable plans may include channel count.
- Each service account stores plan, installation address, activation date, billing start date, billing/due day, current rate, status and assigned collector.
- Plan price changes must only affect future billing; historical invoices preserve billed rates.

3.4 Billing and Invoice Engine

- Generate monthly invoices by active service account.
- Support DRAFT, UNPAID, PARTIALLY_PAID, PAID, OVERDUE, VOID and CREDITED states.
- Invoice details may include subscription charge, installation/reconnection fee, discount, penalty if enabled, debit/credit adjustments and total.

- Finalized invoices are immutable except through controlled adjustment/void workflows.
- Prevent accidental duplicate billing for the same service account and billing period.

3.5 Subscriber Ledger

Date	Reference	Description	Debit	Credit	Balance
2026-09-01	INV-1001	September Internet	999.00		999.00
2026-09-05	RCPT-5001	Cash Payment		500.00	499.00
2026-09-20	RCPT-5110	GCash Payment		499.00	0.00

The ledger must provide a complete chronological debit/credit history and a reproducible running balance. A Statement of Account must be printable for a selected date range.

3.6 Payments and Allocation

- Support Cash, GCash, Bank Transfer, Cheque and Other payment methods.
- Capture payment date/time, amount, receipt number, actor, payment method, reference number, notes and optional proof attachment.
- Support exact, partial and advance payments.
- Default allocation rule: oldest unpaid invoice first.
- Allow authorized manual allocation only when the business rule permits it.
- Payments already posted must be corrected by reversal, not destructive editing.

3.7 GCash Verification

1. Customer sends proof of successful GCash transaction to the Bukidnon Cable and Internet Services Facebook Page.
2. Staff records the transaction reference, sender details, amount and proof image.
3. System checks for possible duplicate references.
4. Authorized staff verifies or rejects the payment proof.
5. Only verified payments are posted and allocated to subscriber invoices.
6. Verification identity and timestamp are retained for audit.

Important

A screenshot is evidence to review, not automatic proof of payment. The application must not mark the account paid solely because an image was uploaded.

3.8 House-to-House Collection

- Define collection areas/routes and assign subscribers to collectors.
- Generate printable route sheets showing account, address, current bill, arrears and total due.
- Create collection batches with OPEN, IN_PROGRESS, SUBMITTED, REMITTED, RECONCILED and CLOSED states.
- Summarize expected receivable, Cash collected, GCash/non-cash collected and uncollected amounts.
- Reconcile collector remittance and explicitly record shortage/overage when totals differ.
- Require authorized confirmation before closing a reconciled batch.

3.9 Receivables and Overdue Monitoring

- Dashboard totals for current receivable, overdue receivable, subscribers overdue and accounts for follow-up/suspension.
- Accounts Receivable aging buckets: Current, 1–30, 31–60, 61–90 and 90+ days.
- Overdue list must show subscriber, service, area, collector, months unpaid, oldest unpaid invoice, last payment and total arrears.
- Allow filters by collector, area, plan, service type and delinquency age.

3.10 Suspension and Reconnection

- Maintain configurable grace period and suspension threshold.
- Record suspension reason, effective date, approved by and notes.
- After qualifying payment, create reconnection workflow with fee where applicable, technician assignment, request date and completion date.
- All service state changes remain in service history.

3.11 Receipts and Reports

- Unique receipt numbers; voided receipts remain in history and numbers are never reused.
- Daily, weekly, monthly and annual collection reports.
- Cash/GCash/payment-method summaries.
- Billing versus collection, revenue by plan/service/area and AR reports.
- Subscriber master list, subscriber ledger and Statement of Account.
- Collector assignment, collection, remittance, shortage/overage and performance reports.
- Payment adjustment, reversal, voided receipt, user activity and audit reports.
- Export applicable reports to PDF and XLSX; CSV may be included where useful.

4. Non-Functional, Security and UI/UX Requirements

4.1 Professional Modern UI/UX Standard

The interface must look like a professional commercial ISP billing and operations platform: clean, trustworthy, efficient, information-dense, and consistent. Avoid generic dashboard templates, excessive gradients, oversized decorative cards, animated counters, and unrelated visual effects.

Design Token	Recommended Value / Rule
Primary / Navy	#0F2747
Accent Blue	#2563EB
Canvas	#F6F8FB
Surface	#FFFFFF
Text Primary	#0F172A
Text Secondary	#64748B
Success	#059669
Warning	#D97706

Design Token	Recommended Value / Rule
Danger	#DC2626
Typography	Inter, Geist, or a comparable professional sans-serif
Financial Values	Right-aligned; tabular numerals where available
Status Indicators	Use text + color, never color alone

4.2 Required Desktop Navigation

```

Dashboard
Subscribers
├─ All Subscribers
├─ New Subscriber
└─ Service Accounts
Billing
├─ Current Billing
├─ Generate Billing
└─ Invoices
Payments
├─ Receive Payment
├─ Payment History
└─ GCash Verification
Collections
├─ Collectors
├─ Areas & Routes
├─ Collection Batches
└─ Remittance
Receivables
├─ Outstanding
├─ Overdue
├─ Aging
└─ Suspension Candidates
Services
Reports
Administration

```

4.3 Screen-Level Requirements

- Dashboard: 4–6 compact KPIs, billing vs collection, payment method summary, AR aging, collector performance, overdue alerts and recent payments.
- Subscriber Profile: overview, services, billing, payments, ledger, collection, service history, documents and audit tabs.
- Receive Payment: optimize for cashier speed—search, balance, amount, method, allocation preview, post, print receipt.
- GCash Verification: two-pane queue with proof preview and verification controls.
- Collector Reconciliation: expected Cash, remitted Cash, difference, non-cash totals, accounts collected and exceptions.
- Operational tables: sticky headers, search, filters, sorting, pagination/virtualization, export and consistent badges.
- Forms: visible labels, required indicators, section grouping and inline validation.
- Printing: dedicated clean print layout without application navigation.

4.4 Security and Audit Requirements

- Username/password authentication with secure password hashing and account active/inactive state.
- RBAC with granular permissions such as subscriber.view, billing.generate, payment.create, payment.reverse, collection.reconcile, report.export, user.manage and backup.restore.
- Server-side authorization is mandatory; hiding a button in React is not authorization.
- All posted financial mutations create an audit record with actor, action, date/time, reason and relevant old/new values.
- Audit logs must not be editable through normal application screens.
- Validate uploaded payment proofs by type, size and safe storage path.
- Do not log passwords, secrets or full confidential payment credentials.
- Provide session lock/logout and sensible failed-login handling.

4.5 Reliability and Performance

- Target at least 20,000 subscribers, 500,000 invoices, 500,000 payments and 1,000,000 ledger-related entries.
- Use indexes and paginated server queries rather than loading full datasets into the renderer.
- Critical financial operations must be transactional: either all database changes succeed or none do.
- Provide database/attachment backup, backup verification and restore workflow.
- Application errors must produce understandable user messages while technical details go to structured logs.

5. Minimum Data Model

The schema may be refined, but the implementation must preserve these business concepts and relationships.

Domain	Required Tables / Entities
Security	users, roles, permissions, role_permissions, user_roles
Subscribers	subscribers, subscriber_addresses/contact data
Services	service_types, service_plans, service_accounts, service_events/devices as needed
Billing	billing_cycles, invoices, invoice_items, adjustments
Payments	payments, payment_allocations, payment_proofs, payment_reversals
Collections	collection_areas, collector_assignments, collection_batches, batch_accounts, collector_remittances
Documents	receipts / receipt records
Operations	suspension_records, reconnection_records
System	audit_logs, application_settings, backup_history

5.1 Required Relationships

```
Subscriber 1 —< ServiceAccount 1 —< Invoice 1 —< InvoiceItem
Invoice >—< Payment (through PaymentAllocation)
Collector 1 —< CollectionBatch 1 —< CollectionBatchAccount
Payment 1 —< PaymentProof / PaymentReversal
ServiceAccount 1 —< ServiceEvent / Suspension / Reconnection
```

5.2 Minimum Constraints

- Unique subscriber account number.
- Unique service account number.
- Unique invoice number and protection against duplicate billing period/service combinations.
- Unique receipt number; voided numbers remain reserved.
- GCash reference uniqueness or controlled duplicate-detection rule.
- Foreign keys for all owned records.
- Indexes on subscriber name/account/contact, invoice due date/status, payment date/reference, collector and collection area.

6. Laboratory Procedure and Development Phases

Recommended execution

Complete the activity incrementally. Each phase has a checkpoint. The instructor may require Git tags/releases or screenshots at every checkpoint. Do not postpone all testing until the final week.

Phase 1 — Analysis and Project Foundation

Required Work	Checkpoint Evidence
Analyze the scenario; establish Git repository/monorepo; configure Electron, React, TypeScript strict, Fastify, PostgreSQL, Drizzle migrations, environment handling, linting and testing.	Architecture diagram; initialized repository; database connection; health endpoint; Electron shell; README.

Phase 2 — Authentication and RBAC

Required Work	Checkpoint Evidence
Implement users, roles, permissions, login, logout/session lock, protected API routes and permission-aware navigation.	Working login; seeded admin; role matrix; authorization tests.

Phase 3 — Plans, Subscribers and Service Accounts

Required Work	Checkpoint Evidence
Implement plan management, subscriber registration, addresses, service accounts, status history, assignment to area/collector and fast search.	CRUD workflows with validation; subscriber profile; service account screen; sample records.

Phase 4 — Billing and Ledger Engine

Required Work	Checkpoint Evidence
Implement monthly billing, invoice numbering, line items, duplicate-billing prevention, due dates, invoice states and ledger debit posting.	Billing generator; invoice view; ledger entries; billing unit/integration tests.

Phase 5 — Payments, Allocation and Receipts

Required Work	Checkpoint Evidence
Implement Cash/GCash/etc., oldest-first allocation, partial and advance payment handling, receipts, proof attachments, reversals and audit trail.	Payment screen; allocation preview; receipt; GCash verification; reversal workflow; tests.

Phase 6 — Collector and Remittance Management

Required Work	Checkpoint Evidence
Implement collection areas, routes, collector assignments, batches, printable route sheet, collection recording, remittance, shortage/overage and reconciliation.	Collection batch lifecycle; reconciliation screen; collector report; tests.

Phase 7 — Receivables and Service Control

Required Work	Checkpoint Evidence
Implement outstanding/overdue lists, AR aging, follow-up filters, suspension candidates, suspension records and reconnection workflow.	Aging report; overdue dashboard; suspension/reconnection history.

Phase 8 — Reports, Dashboard and Printing

Required Work	Checkpoint Evidence
---------------	---------------------

Required Work	Checkpoint Evidence
Implement management KPIs, monthly collection reports, revenue/AR/collector reports, Subscriber SOA, PDF/XLSX export and print layouts.	Dashboard; at least 6 required reports; PDF/XLSX evidence.

Phase 9 — Backup, Hardening and Deployment

Required Work	Checkpoint Evidence
Implement backup/restore procedure, integrity check, error logging, security review, LAN deployment and three-PC concurrency validation.	Backup/restore test; installer; deployment guide; three-client concurrency test.

Phase 10 — QA, Documentation and Defense

Required Work	Checkpoint Evidence
Run complete acceptance test suite, resolve defects, prepare technical documentation, user manual and live demo.	Final source; test report; documentation; demo presentation; release build.

7. Mandatory Acceptance Tests

All tests below are required. A feature may appear visually complete but is considered incomplete if the financial result is incorrect.

ID	Scenario	Test Data	Expected Result
AT-01	Exact payment	Invoice ₱999; payment ₱999	Remaining ₱0; invoice PAID; ledger balanced; receipt created.
AT-02	Partial payment	Invoice ₱999; payment ₱500	Remaining ₱499; PARTIALLY_PAID; allocation and ledger correct.
AT-03	Advance payment	Monthly ₱1,000; payment ₱3,000	Allocation/credit follows documented policy without losing value.
AT-04	Oldest-first arrears	August ₱999 + September ₱999; payment ₱1,200	August ₱0; September ₱798 remaining.
AT-05	Duplicate GCash reference	Post an already-used verified GCash reference	System blocks or explicitly warns according to documented verification policy.
AT-06	Payment reversal	Reverse an incorrectly posted payment	Original remains visible; linked reversal exists; balances restore; actor/reason audited.
AT-07	Collector balanced remittance	Cash collected ₱20,000; remitted ₱20,000	Difference ₱0; batch may reconcile/close.
AT-08	Collector shortage	Cash collected ₱20,000; remitted ₱19,500	₱500 shortage displayed; cannot silently close as balanced.

ID	Scenario	Test Data	Expected Result
AT-09	Concurrent users	Two/three PCs post/read different valid operations	No data corruption, duplicate numbering or unauthorized cross-session effect.
AT-10	Authorization	Cashier attempts admin-only user/backup operation	Server rejects operation even if API is called directly.
AT-11	Duplicate billing	Run billing generation twice for same period	No duplicate finalized invoice for same service account/period.
AT-12	Backup/restore	Create test backup, change data, restore approved backup	Restored database passes integrity check and expected records return.

7.1 Required Test Evidence

- Automated test output for domain/business rules.
- API/integration test results for posting flows.
- E2E or reproducible manual test evidence for key desktop workflows.
- Screenshots or report files showing expected balances.
- Bug log showing at least discovered issue, root cause, fix and regression test where appropriate.

8. Minimum Demonstration Dataset

Seed realistic data rather than demonstrating with only one subscriber. Minimum recommended demo data:

Data Type	Minimum
Users	5 users covering at least Admin, Cashier, Collection Supervisor, Auditor/Viewer
Plans	3 Internet, 2 Cable, 2 Combo
Subscribers	50
Service Accounts	60+ across Internet/Cable/Combo
Collectors	2
Collection Areas	3
Invoices	At least 3 billing months across the dataset
Payments	Mix of Cash, GCash, partial, exact and advance payments
Overdue Accounts	At least 10 spanning multiple aging buckets
Reversal / Void	At least 1 approved example
Suspension / Reconnection	At least 2 scenarios

Data privacy

Use synthetic/demo subscriber information for laboratory work. Do not use real customer names, phone numbers, GCash details, passwords, or production databases.

9. Required Deliverables

Deliverable	Required Contents
D1 — Source Repository	Complete source, meaningful Git history, .gitignore, environment example, migrations and seed scripts.
D2 — Working Release	Installable/run-ready desktop client plus documented API/server and PostgreSQL setup.
D3 — Technical Documentation	Requirements, architecture, module description, ERD/data dictionary, API/IPC design, security model, deployment and backup/restore.
D4 — User Manual	Login, subscriber registration, billing, payment, GCash verification, collection, reports, backup and common troubleshooting.
D5 — Testing Package	Test plan, automated test outputs, acceptance-test results and defect/fix evidence.
D6 — Reports	Sample PDF/XLSX monthly collection report, AR aging, subscriber SOA, collector remittance/performance and other required reports.
D7 — UI Evidence	Screenshots of dashboard, subscriber profile, payment, GCash, collection reconciliation, overdue/aging and reports.
D8 — Presentation / Defense	Architecture explanation, live workflow demo, financial test demo, security explanation and limitations/future work.

9.1 Submission Folder Structure

```

BCIS-Subscription-Billing-System/
├── source/
├── database/
│   ├── migrations/
│   └── seeds/
├── docs/
│   ├── technical-documentation.pdf or .docx
│   ├── user-manual.pdf or .docx
│   ├── erd.png or .pdf
│   └── deployment-guide.md
├── tests/
│   ├── acceptance-test-report.pdf
│   └── screenshots/
├── reports-samples/
├── release/
└── README.md

```

10. Grading Rubric — 100 Points

Criterion	Pts	Performance Standard
A. Requirements & Architecture	10	Clear business analysis, correct 3-PC client-server architecture, sound module boundaries, documented decisions.
B. Database & Data Integrity	15	Normalized/appropriate schema, migrations, constraints, indexes, transactions, correct historical preservation.
C. Billing, Ledger & Payment Correctness	20	Invoices, partial/advance payments, allocation, balances, receipts, reversals and ledger behave correctly.
D. Collection & Receivables	12	Collector batches, remittance, reconciliation, overdue monitoring, AR aging and service-control workflows work.
E. UI/UX & Professional Design	12	Modern consistent design, fast cashier workflow, good tables/forms/statuses, responsive desktop behavior and print layouts.
F. Security, RBAC & Audit	10	Authentication, server authorization, validation, safe Electron boundary, audit trail and safe attachments.
G. Reports & Management Dashboard	8	Useful KPIs, required reports, accurate calculations, PDF/XLSX output.
H. Testing & Quality	8	Mandatory acceptance tests, automated coverage of financial rules, error handling and regression evidence.
I. Documentation & Deployment	3	Complete technical/user/deployment/backup documentation.
J. Demonstration & Code Understanding	2	Architecture, code, and business rules are explained clearly; the demonstration is controlled and credible.

10.1 Critical Failure Conditions

Grading safeguard

The instructor may cap the score or require rework if any critical condition exists: destructive deletion of posted payments, obviously incorrect ledger balances, shared database-file architecture for simultaneous PCs, plaintext passwords, renderer direct database access, no authorization checks on the server, inability to restore a backup, or a submission that is only static UI/CRUD without the required financial workflows.

11. Final Demonstration and Defense

11.1 Required Live Demo Sequence

1. Log in as an administrator and show the dashboard.
2. Create or open a subscriber with Internet/Cable service account.
3. Generate or display the monthly invoice and ledger debit.
4. Post an exact or partial Cash payment and print/preview the receipt.

5. Post/verify a GCash payment with proof and show duplicate-reference protection.
6. Show the subscriber ledger and Statement of Account.
7. Open an overdue subscriber and demonstrate AR aging/filtering.
8. Create/open a collector batch, record collections, remit Cash and reconcile the difference.
9. Demonstrate a payment reversal and the resulting audit trail.
10. Show at least one management report exported to PDF/XLSX.
11. Demonstrate role restrictions using a lower-privileged account.
12. Show backup creation and explain the tested restore procedure.
13. Demonstrate simultaneous operation from multiple office clients if laboratory infrastructure allows it.

11.2 Defense Questions

- Why was PostgreSQL selected instead of a shared SQLite database for three PCs?
- Where does authoritative billing/payment logic live, and why?
- How do you prevent duplicate invoices, duplicate receipts and duplicate GCash transactions?
- How is a partial payment represented in the database?
- Why are posted payments reversed instead of deleted?
- How do you prove the ledger balance is correct?
- What happens if the API crashes halfway through a payment posting operation?
- How does server-side authorization differ from hiding buttons in the UI?
- How are backups validated and restored?
- What design changes would be required for a future collector mobile/PWA application?

12. AI-Assisted Development and Academic Accountability

AI coding assistants such as Claude Code may be used when permitted by the instructor. AI use does not reduce the requirement to understand, test, document, and defend the submitted work.

- Keep project-level development instructions in CLAUDE.md and reusable project rules/skills under .claude/.
- Ask AI to implement feature-sized tasks rather than “build the whole system” in one prompt.
- Review generated migrations, SQL, security-sensitive code and financial calculations before accepting changes.
- Never provide production passwords, real subscriber databases, real GCash details or other private customer information to an AI tool.
- Retain meaningful Git history showing development and correction over time.
- During the defense, the developer(s) must be able to explain the submitted code and design decisions. “The AI generated it” is not an acceptable technical explanation.

12.1 Recommended AI Development Workflow

```

Read requirement
↓
Inspect existing architecture
↓
Identify affected domain/data/security rules
↓
Implement smallest coherent feature
↓
Add/update migrations and validation
↓
Add tests
↓
Run typecheck + lint + tests
↓
Review diff for financial/security impact
↓
Commit with a meaningful message

```

13. Suggested Instructor Checkpoints

Checkpoint	Evidence to Inspect	Suggested Weight / Status
CP1 — Architecture	Repository, diagram, stack, database connection, Electron shell	Formative
CP2 — Core Data	RBAC, plans, subscribers, service accounts, migrations	Formative
CP3 — Financial Core	Billing, ledger, payment allocation tests	Major checkpoint
CP4 — Collections	Collector batches, remittance, AR aging	Major checkpoint
CP5 — Reports/Security	Reports, audit, permission tests, backup	Formative
CP6 — Final QA	Acceptance suite, release, documentation, defense	Summative

14. Final Submission Checklist

- ☐ Application starts without development-only manual edits.
- ☐ All three client PCs can connect to the API/database environment.
- ☐ No secrets or passwords are committed to the repository.
- ☐ Database migrations run from a clean database.
- ☐ Seed/demo data is synthetic.
- ☐ RBAC and server-side permission checks are demonstrated.
- ☐ Monthly billing does not create duplicates.
- ☐ Partial, exact and advance payment cases are correct.
- ☐ Payment reversal preserves history and audit trail.
- ☐ Receipt numbers are unique and voided numbers are not reused.
- ☐ Collector shortage/overage is visible and not silently balanced.
- ☐ AR aging totals reconcile to outstanding invoice balances.
- ☐ PDF/XLSX reports calculate correctly.

- ☐ Backup and restore were tested.
- ☐ Unit/integration/E2E or equivalent acceptance evidence is included.
- ☐ Technical documentation, user manual and deployment guide are included.
- ☐ Final release/installer and README are included.
- ☐ The implementation and design decisions can be explained during the defense.

15. Optional Enhancement Tasks

- Collector mobile/PWA with offline queue and controlled synchronization.
- SMS/email payment reminders using an approved provider.
- Customer self-service portal and digital Statement of Account.
- Official online payment gateway integration.
- QR-coded receipts and subscriber account lookup.
- GIS subscriber/service-area mapping.
- Technician work orders and network equipment inventory.
- Owner analytics for churn risk, revenue forecasting or route optimization.

Bonus rule

Optional features should only receive bonus/recognition after the mandatory financial, security, testing and documentation requirements are complete and correct.

16. Completion Standard

A successful submission behaves like a small production business system. It must preserve financial history, correctly compute balances, support three concurrent office users, prevent unauthorized actions, reconcile collectors, expose overdue receivables, and produce trustworthy reports. Visual polish is required, but it cannot compensate for incorrect financial logic or weak data integrity.

END OF LABORATORY ACTIVITY